

Regular Expressions Cheat Sheet

One pattern language for grep, sed and awk: what each symbol matches, and which tools understand it.

Using it

```
grep -E 'pattern' file    ← -E = extended regex: + ? | ( ) { } work without backslashes
sed -E 's/pattern/new/' file ← sed takes -E too (older GNU sed: -r)
awk '/pattern/ { print }' file ← awk always uses extended regex
```

Characters and classes

Write	Matches	Write	Matches
.	Any one character	<code>[[:digit:]]</code>	A digit, same as <code>[0-9]</code>
<code>[abc]</code> / <code>[a-z]</code>	One of a, b, c / a lowercase letter	<code>[[:alpha:]]</code>	A letter (<code>[[:upper:]]</code> , <code>[[:lower:]]</code>)
<code>^[abc]</code>	Any character except a, b, c	<code>[[:alnum:]]</code>	A letter or digit
<code>\w</code> / <code>\s</code>	Word char / whitespace (GNU tools)	<code>[[:space:]]</code>	Space, tab or other whitespace
<code>\d</code>	A digit, only in grep -P, perl, python	<code>[[:xdigit:]]</code>	A hex digit: 0-9, a-f, A-F

Anchors, repeats and groups

Write	Matches	Write	Matches
<code>^</code> / <code>\$</code>	Start / end of the line	<code>*</code>	The previous item, 0 or more times
<code>\b</code>	A word boundary (GNU, -P, python)	<code>+</code>	1 or more times
<code>\<</code> / <code>\></code>	Start / end of a word (GNU)	<code>?</code>	0 or 1 time (optional)
<code>(ab)+</code>	() groups: ab, abab, ababab	<code>{3}</code>	Exactly 3 times
<code>(cat dog)s</code>	means or: cats or dogs	<code>{2,}</code>	2 or more times
<code>(\w+) \1</code>	\1 = what group 1 matched	<code>{2,5}</code>	2 to 5 times

Greedy: `<. *>` on `<a><b>` grabs the whole thing; `<[ ^> ] *>` stops at the first `>` (or `<. *?>` with grep -P).

Basic (BRE) vs extended (ERE)

Feature	BRE / ERE	Tool	Uses by default
One or more / optional	<code>\+</code> <code>\?</code> / <code>+</code> <code>?</code>	grep, sed	BRE (basic)
Or	<code>\ </code> / <code> </code>	grep -E, sed -E	ERE (extended)
Group	<code>\(\)</code> / <code>()</code>	awk	ERE (always)
Count	<code>\{2,5\}</code> / <code>{2,5}</code>	grep -P, perl, python	PCRE (Perl style, has <code>\d</code>)
<code>.</code> <code>*</code> <code>[]</code> <code>^</code> <code>\$</code>	Same in both	grep -F	No regex at all: plain text

Escaping: `\` makes a special character literal: `1\.5`, `\$HOME`, `\[error\]`. In BRE, plain `+` `?` `|` `( )` `{ }` are literal.

Common patterns (tested with grep -E)

Finds	Pattern	Note
IPv4 address	<code>([0-9]{1,3}\.){3}[0-9]{1,3}</code>	Also accepts 999.1.1.1
Email-ish	<code>[[:alnum:] . % + -] + @ [[:alnum:] . -] \. [[:alpha:]] {2,}</code>	Rough, fine for logs
Blank line	<code>^\$</code>	<code>^[[:space:]]*\$</code> : spaces too
Comment line	<code>^[[:space:]]*#</code>	GNU short form: <code>^\s*#</code>
Trailing whitespace	<code>[[:space:]]+</code>	
Date YYYY-MM-DD	<code>[0-9]{4}-[0-9]{2}-[0-9]{2}</code>	Doesn't check ranges
MAC address	<code>([[:xdigit:]]{2}:){5}[[:xdigit:]]{2}</code>	e.g. 00:1a:2b:3c:4d:5e
Word at line start	<code>^ERROR\b</code>	ERROR first, not ERRORS

Handy recipes

Command	Does
grep -vE <code>'^[[:space:]]*(# \$)' /etc/ssh/sshd_config</code>	Shows only active settings
sed -Ei <code>'s/[[:space:]]+\$//'</code> file	Strips trailing spaces (GNU sed)
grep -oE <code>'([0-9]{1,3}\.){3}[0-9]{1,3}'</code> log sort uniq -c	Counts how often each IP appears

Remember: use single quotes, and **grep -E** for anything beyond simple matches.